

Hybrid Claude Agent SDK + MCP Architecture

Production patterns for layered simulator systems on Replit

Architecture research artifact (regenerated 2026-04-29)

April 28, 2026

Hybrid Claude Agent SDK + MCP Architecture

Executive summary

1. FastMCP composition architecture
 2. Sanitize / synthesize gate pattern
 3. Layer-specific agent policy
 4. Claude Agent SDK integration
 5. Replit deployment constraints
 6. Prompt caching policy
 7. Anti-pattern catalog
 8. Sampling channel preservation rules
 9. Adaptive gate policy
 10. Concrete code patterns
 11. What's likely to change
- Appendix: AGENTIC_ARCHITECTURE_INDEX.md skeleton

Hybrid Claude Agent SDK + MCP Architecture

Executive summary

Five architectural decisions, in order of cost-of-being-wrong:

1. **FastMCP composition uses `mount()`, never chained `Client` calls.** Mounting preserves a single transport, which preserves the bidirectional sampling channel. Chaining clients introduces a process boundary that the MCP `sampling/createMessage` request cannot cross — sampling fails silently.
2. **Streamable HTTP transport must run in stateful mode on Replit.** Stateless mode disables sampling, elicitation, and roots — three capabilities the agentic layers depend on. Stateful mode requires `Mcp-Session-Id` headers and a Redis-backed EventStore for resumability across replica restarts.
3. **The synthesizer is the sole writer to canonical state.** Agents propose; deterministic sanitizers validate; a single synthesizer writes to disk. This keeps calibrated baselines un-corruptible by hallucination. The architectural cost of a violation is silent drift that surfaces weeks later.
4. **Adaptive gates per layer: strict for params/baselines, soft for derived/insights.** Strict gates demand schema + provenance + calibration-impact verification before write. Soft gates accept agent latitude on text outputs. Cost of an undetected error scales with the layer; gate strictness must scale with it.

5. **Calibration is a hard gate before agentic layers.** The CALIBRATION_LOCKED file is committed to git with SHA256 hashes of every params and baselines yaml. Every agentic milestone runs verify_lock_against_current_state() before starting work and aborts if the hashes drifted. This catches yaml edits that would invalidate downstream agent outputs *before* those outputs ship.

The deterministic core (Layers 1, 2, 3, 6, 7, 9) stays agent-free. Agents live only on Layers 4 (baselines), 5 (curation), 6 (scenario synthesis sublayer), and 8 (insights). Layer 10 (interview tools) is LLM-gated but calibration-locked, not agentic in the deliberation sense.

This document is the architectural spec for milestones M12 through M18 of the Reels AT Simulator build. It is the second of three reference documents Claude Code reads during build-time:

Document	Purpose	Created during
reels_metrics_comprehensive_v2.md	Numerical anchors and calibration targets	M5
architecture_research.pdf (this document)	Agentic architecture patterns and constraints	M12 prelude
REFERENCE_DOC_INDEX.md, AGENTIC_ARCHITECTURE_INDEX.md	Per-document indices for fast lookup	M5, M12

1. FastMCP composition architecture

1.1 The single front-door rule

The MCP specification defines a server as the unit that exposes tools, resources, prompts, and sampling. A **client** establishes a session with a server via a transport (stdio, SSE, or Streamable HTTP). Sampling is defined as a server-initiated request the server sends *to its connected client*.

When the goal is to compose multiple internal services (baselines, curation, engine compiler, insights) into one user-facing endpoint, two patterns are available:

Pattern A — mount() composition. One outer FastMCP server. Internal sub-servers are mounted under namespace prefixes. All tools, resources, and prompts surface through the outer server’s transport. Process memory and event loop are shared.

```
# infra/server.py
from fastmcp import FastMCP

front_door = FastMCP("Reels Hybrid", auth=oauth_provider)

# Sub-servers defined in their own modules:
from baselines.server import baselines_server #
FastMCP("baselines")
from curation.server import curation_server # FastMCP("curation")
from engine.server import engine_server #
FastMCP("engine_compiler")
from insights.server import insights_server # FastMCP("insights")

front_door.mount("l4", baselines_server)
```

```

front_door.mount("l5", curation_server)
front_door.mount("l6", engine_server)
front_door.mount("l8", insights_server)

# All sub-server tools are now reachable via the front-door
transport.
# A baselines tool registered as "extract_metric" surfaces as
"l4_extract_metric".

```

Pattern B — chained Client calls. The outer server holds a Client instance pointing to each sub-server (potentially running in separate processes). The outer server’s tools forward calls via these clients.

```

# DO NOT DO THIS for layers that need sampling:
async def extract_metric(arg):
    async with Client("http://baselines-service:8002/mcp") as inner:
        return await inner.call_tool("extract_metric", {"arg": arg})

```

Pattern B looks reasonable for service isolation but **breaks sampling** in a way that is silent at runtime.

1.2 Why chained Client breaks sampling — the trace

The MCP sampling flow:

1. A tool inside the baselines server needs to call back to the user’s model. It invokes `ctx.sample(messages=[...])` or `ctx.elicit(...)`.
2. FastMCP translates this into a `sampling/createMessage` JSON-RPC request.
3. The request travels from the server **back through the active transport** to the connected client.
4. The connected client is responsible for routing the request to the user’s configured model (the Claude Agent SDK’s orchestrator, in our case) and returning the response.

In Pattern A (`mount()`), the active transport is the front-door’s Streamable HTTP connection to the orchestrator. The sampling request reaches the orchestrator’s Agent SDK client, which routes to Claude. Response flows back along the same path.

In Pattern B (chained Client), the inner server’s “active transport” is the connection to the **outer server’s client wrapper**, not to the orchestrator. The sampling request reaches the outer server, which has no logic to forward it to *its own* upstream transport. The request times out or returns an error. There is no exception by default — sampling just doesn’t work, and tests that don’t exercise sampling pass.

The MCP spec is explicit on this: sampling is a *server-to-client* request and the client is whoever holds the active transport. Composition that introduces an intermediate server breaks the invariant.

1.3 When to use which

Need	Pattern	Rationale
Tools, resources, prompts only	Either	No sampling, no risk
Sampling required	<code>mount()</code> only	Preserves transport identity
Elicitation required	<code>mount()</code> only	Same channel as sampling
Roots negotiation	<code>mount()</code> only	Same channel
Process isolation needed	Chained Client, but no sampling	Trade-off

Different language sub-servers	Chained Client, no sampling	Trade-off
--------------------------------	-----------------------------	-----------

For the Reels simulator build, every agentic sub-server (baselines, curation, engine compiler, insights) needs sampling. **All four use `mount()`.**

1.4 Tool namespacing under `mount()`

When a sub-server is mounted at prefix "l4", its tools are exposed at the front-door as `l4_<tool_name>`. Resources surface as `l4://...`. Prompts get the same prefix.

The mount prefix should match the layer number, not the sub-server's internal name. This makes the layer architecture visible at the wire level — a tool name like `l4_extract_metric` immediately tells a debugger which layer is doing the work.

Name collisions across mounted sub-servers are handled by the prefix. Two sub-servers can both define a tool called `synthesize` because they surface as `l4_synthesize` and `l8_synthesize`.

1.5 `Mount()` does not give up isolation entirely

A common objection: "if I mount, I lose the ability to deploy these as separate services later." This is partially true and partially not.

What you give up: independent scaling, independent deploy cadence, language heterogeneity, blast-radius isolation on crashes.

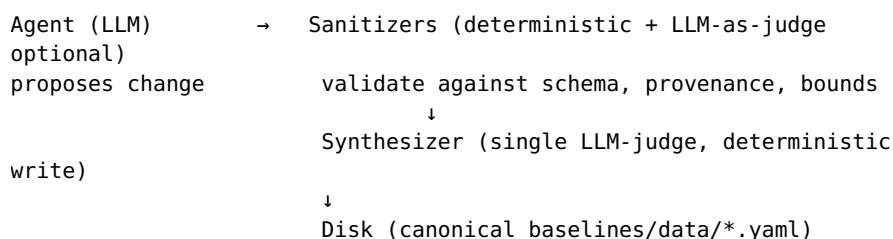
What you keep: code organization (each sub-server is a separate Python module with its own tools and prompts), independent test surfaces (each sub-server can be instantiated standalone in tests), independent ownership (different teams can own different sub-servers).

For the Reels simulator, the deployment target is a Replit Reserved VM with a single Python process. There is no production reason to use process isolation, and several reasons not to (sampling, latency, cost). `Mount()` is the right choice.

2. Sanitize / synthesize gate pattern

2.1 The three-step pattern

The pattern in detail:



Why three steps and not two (skip the sanitizers, let the synthesizer judge directly)?

Sanitizers are cheap; the synthesizer is expensive. Sanitizers run in parallel as code-based or Haiku-based checks. Most rejected proposals are caught here for ~\$0.0001. The synthesizer is an Opus-

4.7 call with thinking budget — ~\$0.10 per invocation. Filtering before the synthesizer reduces synthesizer load by an order of magnitude.

Sanitizers produce structured flags; the synthesizer reasons over them. A schema sanitizer outputs `SanitizerFlag(kind="schema", severity="high", evidence=[...])`. A trust-tier sanitizer outputs another flag. The synthesizer receives the proposal *plus* the accumulated flags and decides accept/reject/escalate. This separates detection (sanitizer’s job) from judgment (synthesizer’s job).

Three steps make the failure mode legible. When something rejects, you can tell whether it was a schema violation, a trust-tier conflict, or the synthesizer’s own judgment. With two steps, you have to introspect the synthesizer’s reasoning to know.

2.2 The four sanitize/synthesize options

The build prompt enumerates four architectural variants. All four are compatible with the three-step pattern; they differ in *what* the steps do.

Option A — Hook-only gate. PreToolUse hook validates before the tool fires. PostToolUse hook annotates the result. No separate synthesizer subagent. Used for: insights narration (Layer 8), where the gate is “did the narrator output hallucinate a number?” rather than “is this baseline correct?”

Implementation complexity	Low
False positive rate	Low
False negative rate	Medium
Token cost	Minimal (hooks are deterministic code)
Latency	Single tool call
When to use	Soft gate, low-stakes outputs

Option B — Sanitizer-as-tool + synthesizer-as-judge. Multiple sanitizer tools run in parallel. Synthesizer is a separate LLM call (Opus 4.7) that reads sanitizer flags and decides. Used for: Layer 4 baselines.

Implementation complexity	Medium
False positive rate	Medium
False negative rate	Low
Token cost	Sanitizer (Haiku) + synthesizer (Opus)
Latency	Parallel sanitizers + serialized synthesizer
When to use	Strict gate, high-stakes writes

Option C — Evaluator-optimizer loop. Compiler proposes; evaluator critiques against rubric; on score below threshold, compiler retries with feedback. Bounded to N iterations. Used for: Layer 6 scenario synthesis.

Implementation complexity	Medium-high
False positive rate	Low (iterates until quality)
False negative rate	Medium (bounded iterations)
Token cost	$N \times (\text{compiler} + \text{evaluator})$
Latency	N round-trips
When to use	Soft gate, exploratory outputs where iteration improves quality

Option D — Constitutional critique-revise. Agent self-critiques against a written constitution (e.g., “always cite primary sources”). Revises proposal in light of critique. Used for: Layer 5 curation.

Implementation complexity	Medium
False positive rate	Medium
False negative rate	Low (constitution is explicit)
Token cost	Single LLM call with critique-revise loop
Latency	Two round-trips (initial + revise)
When to use	Strict gate, where rules are well-articulated

2.3 The synthesizer-as-sole-writer rule

This is the load-bearing architectural constraint. **No agent calls `open(yaml_file, 'w')` directly.** All writes to canonical state go through a `synthesize_*` tool that:

1. Receives the proposed change plus all sanitizer flags.
2. Calls Opus 4.7 with a rubric (factual accuracy, citation accuracy, completeness, source quality).
3. Returns a `SynthesizerDecision(decision, confidence, rationale, accepted_fixes, unresolved_disputes)`.
4. **If decision is accept and confidence $\geq$ floor**, writes to disk (via `aiofiles` with `mtime` tracking).
5. **If decision is escalate**, calls `ctx.elicit()` for human diff-confirm.
6. **If decision is reject**, persists the rejection to the observations table and returns without writing.

What goes wrong when this rule is violated:

- An agent writes a `yaml` directly with a hallucinated value. Calibration breaks two milestones later. Debugging traces back through three commits.
- The provenance audit (M8) catches it on the next CI run, but only because the audit walks every `yaml`. Without the audit, the corruption ships.
- The observations and syntheses tables (the durable artifacts of agentic work) lose their sole-source-of-truth property. You can no longer answer “who decided to change this and why” from the database.

The provenance audit is the enforcement mechanism. Every numeric value in `params/` and `baselines/data/` must carry `{value, source, provenance, confidence}` and the source must resolve in `sources_registry.yaml`. An agent-written value without these fields fails the audit immediately.

2.4 Calibration-impact pre-check

Before any baseline write, the synthesizer runs:

```
from calibration.lock import verify_lock_against_current_state,
propose_baseline_change

# Compute hypothetical post-write state
proposed_state = current_state.with_change(field=value)

# Re-hash the affected file under the proposed state
new_hash = sha256(proposed_state.serialize())
```

```

# Compare to the locked hash
if locked_hashes[file_path] != new_hash:
    # The write would invalidate calibration.
    # Surface to user via elicit:
    ctx.elicit(
        f"This change would invalidate calibration. "
        f"File: {file_path}. "
        f"Re-running calibration may produce different results.
Approve?"
    )

```

Calibration drift detected at the synthesizer level is recoverable. Drift detected after the fact is not — it requires re-running calibration and accepting that downstream agent outputs since the drift may have been wrong.

3. Layer-specific agent policy

Layer	Owns	Agentic?	Mode	Rationale
0	Infra	No	n/a	Boilerplate; deterministic by definition
1	Event substrate	No	n/a	Raw event log; agents have nothing to add
2	Algorithms	No	n/a	Pure functions; agent involvement breaks determinism
3	Parameters	No	n/a	Inputs to algorithms; tuning is calibration's job
4	Baselines	Yes	Strict	Sanitize/synthesize ingestion of external facts
5	Curation	Yes	Strict	Constitutional refresh of sources
6	Engine	Mostly no	Soft (sublayer)	Engine is deterministic; only scenario synthesis sublayer is agentic
7	Metrics DSL	No	n/a	SQL templates; agent-written SQL is unsafe
8	Insights	Yes	Soft	Narration of deterministic detector outputs
9	Calibration	No	n/a	Tests + lock; deterministic
10	Interview tools	LLM-gated	n/a	Uses LLM but calibration-locked, not deliberation-style

3.1 Layer 3 (parameters) — why no agents

The argument for: agents could re-tune parameters based on calibration drift detected in production.

The argument against: parameters are inputs to algorithms. Algorithm semantics depend on stable parameter values. If an agent silently re-tunes `params/ranking_weights.yaml::alpha_watch_time` between runs, the engine's outputs become non-reproducible. Calibration is the *correct* mechanism for parameter tuning — and calibration is deterministic.

Recommendation: Layer 3 stays agent-free. Tuning happens during calibration (M11). If production drift is detected later, the response is to re-run calibration with updated baselines, not to let an agent edit params on its own.

3.2 Layer 4 (baselines) — strict gate, full sanitize/synthesize

Baselines are external facts (Meta DAP, Reels run rate, competitor metrics). They have an authoritative source (SEC filings, earnings transcripts, industry analysts) and a snapshot date. They go stale.

The agentic refresh: 1. **Curation refresher (Layer 5)** identifies which baselines are due for refresh and which sources to query. 2. **Baseline extractor (Layer 4)** uses Files API + Citations API to extract metric values from source documents. 3. **Sanitizers (parallel)** check schema, source tier, and policy. 4. **Synthesizer (Opus 4.7)** weighs sanitizer flags and decides accept/reject/escalate. 5. **Synthesizer writes** to `baselines/data/*.yaml`.

The strict gate: confidence floor 0.85, severity-block threshold “high”, `ctx.elicit()` for any escalation.

3.3 Layer 5 (curation) — constitutional, strict gate

Curation orchestrates baseline refresh and updates `sources_registry.yaml` (the registry that the provenance audit reads).

The constitution is explicit: - Always cite primary sources for numeric values (SEC > earnings_call > analyst > blog). - Never claim unsourced numbers. - Never auto-apply diffs — always require human diff-confirm via `ctx.elicit()`. - Competitor numbers MUST cite the view-definition asymmetry note from `baselines/data/view_definition_asymmetry.yaml`.

The agent may only update baselines and the sources registry. It may not touch `params/`, `algorithms/`, or `engine/`. The `PreToolUse` hook enforces this by blocking any tool call that targets the prohibited paths.

3.4 Layer 6 (engine) — deterministic core, agentic sublayer

The engine itself is deterministic. Same (perturbations, seed) → identical events.

The exception is **scenario synthesis** (Layer 6's sub-layer): translating natural-language hypotheticals into structured perturbation manifests. This is agentic — the user types “what if Meta launched Reels-only mode for teens” and the scenario compiler translates that into a list of {target, op, value} perturbations the deterministic engine can run.

The pattern is evaluator-optimizer (Option C): 1. Compiler proposes a manifest (Sonnet 4.6 with structured outputs). 2. Evaluator critiques (Sonnet 4.6 reading a rubric). 3. On score < 0.7, compiler retries with

feedback. N=2 max. 4. Synthesizer writes to scenarios table; on N=2 exhaustion, writes with disputed=true (soft mode allows exploratory failure).

3.5 Layer 7 (metrics) — never agentic

Metrics are SQL templates. The closed whitelist of group_by dimensions + the parameterized-SQL-only rule make the metric layer safe even with adversarial input. **Letting agents generate SQL violates this safety.**

Three risks: - **SQL injection via dimension whitelisting bypass.** The whitelist exists because string interpolation outside it is unsafe. Agents would routinely propose dimensions outside the whitelist ("creator_id" sounds reasonable until you remember the index doesn't cover it). - **Hallucinated joins.** The substrate has a fixed event schema. An agent that proposes joining to a non-existent table breaks the metric for everyone. - **Non-determinism.** Same metric query with same scenario_id should produce identical results. Agent-generated SQL with different phrasings of the same query produces different execution plans, sometimes different results.

Metrics stay deterministic. New metrics are written by humans (or proposed by agents but reviewed and committed by humans like any other code change).

3.6 Layer 8 (insights) — agentic, soft gate, hook-only

Insights narrate deterministic detector outputs in plain English. The deterministic detectors (insights/deterministic.py) compute z-score anomalies, percent diffs, and ranks. The narrator (insights/server.py) takes those outputs and writes prose.

The constraint: **the narrator MUST NOT invent numeric values.** Every number in the narrator's output must trace back to a value in the deterministic detector's output. The narrator's job is to add context, hypotheses, and language — not data.

The soft gate: - PostToolUse hook annotates the narrator's output with {disputed, confidence, disputed_by} when sanitizer flags exist. - No PreToolUse block — narration is text output, blocking it isn't useful. - The user reads the narration and has agency. If a hypothesis is wrong, the user pushes back; the narrator doesn't ship to production state.

This is the single layer where soft mode is appropriate and strict mode would be over-engineering.

3.7 Layer 10 (interview tools) — LLM-gated, calibration-locked

Interview tools generate questions and evaluate answers. They use Sonnet 4.6 internally but they are **not agentic in the deliberation sense** — there is no sanitize/synthesize pattern, no judge, no observations table.

They are **calibration-locked**: every tool call runs @requires_calibration decorator that asserts calibration/CALIBRATION_LOCKED exists and verify_lock_against_current_state() returns valid. If calibration is unlocked, the tool raises CalibrationLockError.

The rationale: interview questions reference numbers from baselines and metrics. If calibration is invalid, those numbers may not match the simulator's actual behavior. Generating questions against drifted

baselines produces questions that look right but evaluate against wrong answers.

4. Claude Agent SDK integration

4.1 The hooks system

The Claude Agent SDK exposes hooks at three points:

- **PreToolUse** — fires before a tool executes. Can block the call (return `permissionDecision: "deny"`) or modify input.
- **PostToolUse** — fires after a tool returns. Can rewrite the output (`updatedMCPToolOutput`).
- **UserPromptSubmit** — fires when the user sends a message. Can modify the prompt.

For the Reels simulator, hooks enforce architectural rules:

```
# orchestrator/hooks.py

def make_sanitizer_pretooluse_hook(contract):
    """Strict mode: block if confidence below floor or
    severity>high."""
    async def hook(tool_name, tool_input, context):
        if not
        tool_name.startswith(f"{contract.layer}_synthesize_"):
            # Only synthesizer tools may write to canonical state.
            # Non-synthesizer tools are read-only or staging.
            return None # Allow

        # Read sanitizer flags from context (deposited by parallel
        sanitizer tools)
        flags = context.get("sanitizer_flags", [])
        max_severity = max((f.severity for f in flags),
        default="info")
        confidence = context.get("synthesizer_confidence", 0.0)

        if max_severity in ("high", "critical"):
            return {
                "hookSpecificOutput": {
                    "hookEventName": "PreToolUse",
                    "permissionDecision": "deny",
                    "permissionDecisionReason": f"Sanitizer severity
{max_severity}; flags: {flags}",
                }
            }

        if confidence < contract.confidence_floor:
            return {
                "hookSpecificOutput": {
                    "hookEventName": "PreToolUse",
                    "permissionDecision": "deny",
                    "permissionDecisionReason": f"Synthesizer
confidence {confidence} below floor {contract.confidence_floor}",
                }
            }

        return None # Allow

    return hook
```

The hook is the architectural enforcement. Without it, an agent could call any synthesizer tool with any payload. With it, only payloads that pass sanitization reach disk.

4.2 Subagent invocation

A subagent is a programmatic invocation of the Agent SDK with a specific AgentDefinition (model, tools, system prompt, MCP servers). Subagents share the orchestrator's transport and run in the orchestrator's process by default.

Spawn a subagent when: - The work is a clearly bounded task with its own context window. - The task uses a different model than the orchestrator (e.g., orchestrator is Opus, worker is Sonnet for cost). - The task needs different tool permissions than the orchestrator.

Don't spawn a subagent when: - The task is one or two tool calls — just call them in the main agent. - The task is "make a decision based on context" — the synthesizer pattern handles this with a single LLM call, not a subagent.

Token cost: a subagent invocation duplicates the system prompt and tool definitions in its context. With prompt caching at `ttl="1h"`, the cost is amortized across invocations. Without caching, a subagent that's invoked once is roughly 2× the token cost of an inline tool call.

4.3 CLAUDE_CODE_FORK_SUBAGENT=1

This environment variable enables fork-style subagent execution. Forking shares memory pages (copy-on-write) between the parent and child processes, dramatically reducing the per-subagent overhead.

The trade-off: - **With forking:** ~90% input-token savings on fan-out (multiple parallel subagents). Subagents inherit the parent's loaded modules, prompt cache, and connection pools. - **Without forking:** Each subagent loads independently. On Opus, this can mean 60+ seconds of cold-start per subagent.

For Replit Reserved VM deployments with multiple agentic layers, **forking is essentially mandatory**. Set `CLAUDE_CODE_FORK_SUBAGENT=1` in `.replit` and confirm it's exported in the run command.

4.4 Sampling and elicitation

`ctx.sample(messages=[...])` requests a completion from the user's model. `ctx.elicit(prompt)` requests user input. Both are server-to-client requests routed through the active transport.

The full path for `ctx.sample`: 1. Tool inside `baselines/server.py` calls `ctx.sample(messages=[{"role": "user", "content": "..."}])`. 2. `FastMCP` serializes as `sampling/createMessage` JSON-RPC request. 3. Request travels back through the front-door's Streamable HTTP connection. 4. Agent SDK orchestrator receives the request. 5. Orchestrator forwards to Claude (via Anthropic API) using whatever model is configured for the calling agent. 6. Response flows back along the same path.

For this to work end-to-end: - The transport must be stateful (`McpSession-Id` present). - The connection must be alive (keepalives, no idle timeouts). - The orchestrator must declare the sampling capability when it connects. - The mounted sub-server must be reachable through the front-door (no chained Client).

4.5 The deliberation contract

`orchestrator/deliberation.py` defines a `DeliberationContract` per layer:

```
@dataclass
class DeliberationContract:
    layer: int
    mode: Literal["strict", "soft"]
```

```

        confidence_floor: float # synthesizer confidence below this →
block
        severity_block_threshold: str # sanitizer severity at or above
this → block
        escalation_handler: Callable # what to do on escalate decision

```

Layer-to-contract mapping:

```

CONTRACTS = {
    4: DeliberationContract(layer=4, mode="strict",
confidence_floor=0.85,
                                severity_block_threshold="high",
escalation_handler=elicit_human_approve),
    5: DeliberationContract(layer=5, mode="strict",
confidence_floor=0.85,
                                severity_block_threshold="high",
escalation_handler=elicit_human_approve),
    6: DeliberationContract(layer=6, mode="soft",
confidence_floor=0.70,
                                severity_block_threshold="critical",
escalation_handler=write_with_disputed_flag),
    8: DeliberationContract(layer=8, mode="soft",
confidence_floor=0.60,
                                severity_block_threshold="critical",
escalation_handler=write_with_disputed_flag),
}

```

Hooks are mode-agnostic at the call site. They receive the contract for the current layer and apply the appropriate rules. This means adding a new agentic layer is one entry in `CONTRACTS`, not a new hook.

5. Replit deployment constraints

5.1 Stateful Streamable HTTP — mandatory

The MCP spec defines two HTTP transport modes:

- **Stateless:** Each request is independent. No session ID. Server cannot push messages back to client (no sampling, no elicitation, no roots).
- **Stateful:** Sessions are tracked via `Mcp-Session-Id` header. Server can push messages back via the same connection. Sampling, elicitation, roots all work.

For agentic FastMCP servers, **stateful is the only choice that works**. Stateless is a footgun: the server boots fine, tools register fine, but `ctx.sample` returns an error and tests pass anyway because the tests don't exercise sampling.

Replit configuration:

```

# infra/server.py
from fastmcp import FastMCP
from fastmcp.transports import StreamableHTTPSessionManager

front_door = FastMCP("Reels Hybrid", auth=oauth_provider)

# Configure stateful transport
session_manager = StreamableHTTPSessionManager(
    stateless=False, # MUST be False
    event_store=redis_event_store, # see 5.2

```

```

)

# Run on Replit
import os
front_door.run(
    transport="streamable-http",
    session_manager=session_manager,
    host="0.0.0.0",
    port=int(os.environ.get("PORT", 8000)),
)

```

5.2 Redis EventStore for resumability

When a Replit replica restarts (deploy, scaling event, network hiccup), in-process session state vanishes. Without an external EventStore, the client's reconnect loses its session and any pending sampling requests.

The EventStore interface from FastMCP:

```

# infra/eventstore.py
from fastmcp.eventstore import EventStore
import redis.asyncio as redis

class RedisEventStore(EventStore):
    def __init__(self, redis_url: str):
        self.r = redis.from_url(redis_url)

    async def append(self, session_id: str, event: dict) -> str:
        event_id = str(uuid4())
        await self.r.xadd(
            f"mcp:session:{session_id}",
            {"event_id": event_id, "data": json.dumps(event)},
        )
        return event_id

    async def since(self, session_id: str, last_event_id: str) ->
list[dict]:
        # Read events after last_event_id
        ...

```

When the client reconnects with Last-Event-ID: <id>, the session manager pulls events since that ID from Redis and replays them. Sampling requests in-flight at the time of restart resume cleanly.

For Replit deployments, use the upstream Redis (e.g., Upstash) configured via REDIS_URL env var. Local Redis on the same VM is acceptable for development; for production multi-replica, use external Redis.

5.3 Postgres for state, not local SQLite

Replit Deployments may scale or redeploy at any time. **Filesystem writes are not durable across red deploys**. A SQLite file written to ./events.db on one replica disappears when the replica is replaced.

Use Postgres (Replit's managed Postgres or external) for:

- scenarios table
- events table (the M2 substrate)
- observations table (sanitizer flags)
- syntheses table (synthesizer decisions)
- baselines and params mirror tables (for fast lookup; canonical is yaml)
- evaluations table (interview answer evaluations)

SQLite is acceptable in two narrow cases:

- **Tests**. tests/ use SQLite via aiosqlite for speed and isolation. The db.py module uses dialect-variant types so the same code runs against both.
- **Local**

development. Single-developer, single-replica, willing to reset state on redeploy.

Production never uses SQLite.

5.4 Reserved VM vs Autoscale

Need	Choice	Rationale
Sampling latency-sensitive	Reserved VM	Cold-starts on Autoscale add 10-30s
Long-running agentic sessions	Reserved VM	Autoscale may evict mid-session
Cost-conscious development	Autoscale	Pay-per-request
Multi-replica redundancy	Reserved VM $\times$ N	Load balancer in front

For the Reels simulator, **Reserved VM is the right choice**. Sampling latency materially affects user experience, and agentic sessions can run minutes (M12 baseline ingestion, M15 scenario synthesis with N=2 evaluator-optimizer iterations).

5.5 Environment variables

```
# .env.example
MCP_AUTH_TOKEN=<oauth provider token>
ANTHROPIC_API_KEY=sk-ant-...
VOYAGE_API_KEY=pa-...
REDIS_URL=rediss://...
DATABASE_URL=postgresql+asyncpg://...
CLAUDE_CODE_FORK_SUBAGENT=1
CLAUDE_CODE_ENABLE_TELEMETRY=1
```

Replit's secrets manager handles `.env` injection. Never commit secrets to git; the `.env.example` documents the required keys with empty values.

5.6 Health checks and graceful restarts

Replit's health check endpoint pings `/health` on a configurable interval. Implement it:

```
@front_door.app.get("/health")
async def health():
    # Verify Redis reachable
    try:
        await redis_event_store.r.ping()
        redis_ok = True
    except Exception:
        redis_ok = False
    # Verify DB reachable
    try:
        async with db.session() as s:
            await s.execute("SELECT 1")
        db_ok = True
    except Exception:
        db_ok = False
    return {"ok": redis_ok and db_ok, "redis": redis_ok, "db":
db_ok}
```

Graceful restart: when the replica receives SIGTERM, finish in-flight requests (with a timeout), then close transport. The EventStore preserves session state for client reconnect.

6. Prompt caching policy

6.1 The 5-minute default is wrong for agentic workflows

Anthropic's prompt cache defaults to a 5-minute TTL. For chat-style applications this is fine — most conversations are sub-5-minute. For agentic workflows that pause for elicitation (ctx.elicit calls human, human takes 30 seconds to respond), the cache expires mid-flow.

Set "ttl": "1h" explicitly on every cache_control block:

```
client = Anthropic()
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=4096,
    system=[
        {
            "type": "text",
            "text": SYSTEM_PROMPT, # Long static prompt
            "cache_control": {"type": "ephemeral", "ttl": "1h"},
        }
    ],
    messages=[...],
)
```

6.2 What to cache

- **System prompts.** Long, static, used across many invocations. Cache them.
- **Tool definitions.** When a server has 20 tools with detailed schemas, the tool block is large. Cache it.
- **Reference documents.** reels_metrics_comprehensive_v2.md (this document plus the metrics doc) is ~30KB markdown loaded into context for the M12 ingestion step. Cache it.
- **Few-shot example libraries.** The scenario compiler uses 5-10 cached examples. Each is ~500 tokens. Cache them.

6.3 What NOT to cache

- **User-specific content.** Cache misses are cheap; cache pollution from per-user data is expensive.
- **Scenario-specific state.** Each scenario has different perturbations and seeds. Don't cache.
- **Recent tool results.** Tools return JSON that changes per call. Don't cache.

6.4 Cache invalidation

After a baseline refresh (Layer 4 synthesizer commits a change to baselines/data/), bust the cache for any prompt that includes baseline content:

```
# After synthesize_baseline writes to disk:
await invalidate_cache_for_prefix("system_prompt_with_baselines")
```

Cache invalidation is implemented by changing the cached content (e.g., appending a version string to the system prompt). The cache key is the content hash; changing the content changes the key.

6.5 Cost model

- Cache reads: 10% of base input price.

- Cache writes: $1.25 \times$ base input price.
- **Break-even:** ~ 5 reads per write. After 5 reads, the cache pays back; after 50, savings are $>80\%$.

For systems with high re-use (many invocations of the same agent with the same system prompt), savings are dramatic. For one-shot tools, caching is net-negative — don't cache.

7. Anti-pattern catalog

The “do not” list, paired with the file or layer where the temptation arises.

7.1 DO NOT chain MCP client calls when sampling is needed

Temptation arises in: `infra/server.py` initial composition. **Why it fails:** The inner server's `ctx.sample` request reaches the outer server, which has no logic to forward it to the orchestrator's transport. Sampling silently fails. **The right pattern:** `mount()` composition. Single transport, sampling preserved.

7.2 DO NOT use stateless HTTP transport on Replit

Temptation arises in: `infra/server.py` transport configuration. **Why it fails:** Stateless transport disables sampling, elicitation, roots. Tests that don't exercise these capabilities pass anyway. **The right pattern:** `stateless=False` on `StreamableHTTPSessionManager`, `Mcp-Session-Id` header, Redis EventStore.

7.3 DO NOT combine Citations API + Structured Outputs in one call

Temptation arises in: `baselines/tools/extract_metric.py`. **Why it fails:** Returns 400. The two features are mutually exclusive in a single API call. **The right pattern:** Two-call sequence. Call 1: Citations API to gather cited evidence. Call 2: Structured outputs (with `strict:true` tool use) to format the extracted metric.

7.4 DO NOT use Opus on every layer

Temptation arises in: `orchestrator/agents.py` model assignment. **Why it fails:** Cost. Opus 4.7 is $\sim 5 \times$ Sonnet 4.6 at the same token volume. Workers don't need Opus. **The right pattern:** Sonnet 4.6 for extraction, narration, evaluation. Haiku 4.5 for cheap sanitizers. Opus 4.7 only for orchestrator and final synthesis judgment.

7.5 DO NOT have agents on Layers 1, 2, 3, 6 (engine), 7, 9

Temptation arises in: “wouldn't it be nice if an agent could tune ranking weights based on observed behavior?” **Why it fails:** Corrupts the calibrated core. Algorithm semantics depend on stable parameter values; metrics depend on stable SQL; calibration depends on deterministic engine. **The right pattern:** Agents on Layers 4, 5, 6 (scenario synthesis only), 8. Everything else stays deterministic.

7.6 DO NOT let agents call `open()` on yaml files

Temptation arises in: Any tool that “needs” to write a value to a yaml file as part of its job. **Why it fails:** Bypasses the sanitize gate. The synthesizer is the sole writer. **The right pattern:** All writes go through `synthesize_*` tools. Sanitizers are read-only. The provenance audit catches violations on the next CI run, but only if the violation produced a malformed value.

7.7 DO NOT skip the calibration lock verify on agentic milestones

Temptation arises in: “M12 just creates new files in `baselines/server.py`, why would I need the lock check?” **Why it fails:** If `params/` or `baselines/data/` drifted since calibration locked (from a prior agent run, a manual edit, a merge), the agent’s outputs are based on uncalibrated state. The drift may not surface until M14 or later. **The right pattern:** Every agentic milestone runs `verify_lock_against_current_state()` as the first step. Abort on drift.

7.8 DO NOT use multi-agent for tightly-coupled tasks

Temptation arises in: “I’ll spawn three subagents in parallel to handle these three sanitizers.” **Why it fails:** The three sanitizers are functions of the same input and produce flags consumed by the same downstream synthesizer. Parallel subagents add coordination overhead without parallelism gains. **The right pattern:** Multi-agent for genuinely independent subtasks (e.g., “extract metric A from doc 1, extract metric B from doc 2”). For tightly coupled tasks, sequential or parallel-tool-call within one agent.

7.9 DO NOT trust agent-generated SQL

Temptation arises in: Layer 7 metrics, Layer 8 insights (“can the narrator just write a custom query?”). **Why it fails:** SQL injection (even with parameterization, agents propose interpolated dimension names), hallucinated joins, non-determinism. **The right pattern:** Metrics are human-authored SQL templates. The compiler enforces a closed dimension whitelist. Agents may *select* metrics; they may not generate them.

7.10 DO NOT rely on the 5-minute cache default

Temptation arises in: Forgetting to set `"ttl": "1h"` because the default mostly works. **Why it fails:** Agentic workflows that pause for `ctx.elicit` exceed 5 minutes. Cache expires mid-flow. Subsequent calls reload the entire system prompt. **The right pattern:** `"ttl": "1h"` explicit on every `cache_control` block.

7.11 DO NOT nest tool calls more than 3 deep

Temptation arises in: Composed agentic flows where Tool A calls subagent that calls Tool B that calls Tool C. **Why it fails:** Context window pressure (each level adds tool definitions and history), debugging difficulty (which level produced the error?), latency multiplication. **The right pattern:** Flatten. If your flow is 4+ deep, the architecture has the wrong decomposition. Sanitizers are parallel siblings of the agent, not nested under it.

8. Sampling channel preservation rules

The sampling channel is the mechanism by which an MCP server can request a completion from the user's model. Preserving this channel is the single most important architectural concern in the agentic layers.

8.1 The sampling request flow

1. Tool inside an agentic server (e.g., `baselines/tools/extract_metric.py`) calls `ctx.sample(messages=[...])`.
2. FastMCP serializes the call as a `sampling/createMessage` JSON-RPC request.
3. The request travels **back through the active transport** to the connected client.
4. The connected client (Agent SDK orchestrator) receives the request and routes to the user's configured model.
5. Response flows back through the same transport.

8.2 What breaks the channel

- **Stateless transport.** The server has no way to push messages back. Sampling requests have nowhere to go.
- **Chained client calls.** Pattern B from §1.1. The intermediate server is not the original client; it has no transport pointing to the orchestrator.
- **Process isolation without channel forwarding.** A subprocess server with its own transport has a transport, but it points to its parent process, not to the orchestrator. Without explicit forwarding (which FastMCP doesn't provide), sampling fails.
- **Dead transport connections.** A connection that's been idle long enough to be reaped by a load balancer or proxy. Replit's default proxy idle timeout is 60s; configure keepalives.
- **Missing capability declarations.** The client must declare it supports sampling during the MCP handshake. If the orchestrator's Agent SDK config omits it, sampling is rejected at the protocol level.

8.3 What preserves the channel

- **Stateful Streamable HTTP.**
- **`mount()` composition.**
- **Explicit capability declarations.** In the Agent SDK orchestrator config: `capabilities: {"sampling": {}}`.
- **Transport keepalives.** WebSocket/SSE pings every 30s.
- **Proper session management.** `Mcp-Session-Id` present on every request, `EventStore` configured for resumability.

8.4 Testing sampling

In development, common silent failures:

- Tool returns `None` from `ctx.sample` instead of a completion. Suggests sampling is reaching the client but the client isn't routing to a model. Check Agent SDK config.
- Tool times out on `ctx.sample`. Suggests the request never reached the client. Check transport mode and composition pattern.
- Tool returns a completion but the content is empty. Suggests the model is being called but with empty messages. Check serialization.

The acceptance test (M18) includes: > "Sampling channel: a layer4 tool successfully calls `ctx.sample()` back to the orchestrator."

This is the canary. If it passes, the channel is intact. If it fails, fix before shipping.

8.5 Cost attribution

Sampling requests bill against the **user's** account, not the server's. The user's API key is the one used for the actual model call. The MCP server is just routing.

This has implications: - Usage tracking must be done at the orchestrator level, not the server level. - Rate limits apply to the user's tier, not the server's. - The user can revoke the API key without affecting the MCP server itself; the server keeps running but sampling fails.

9. Adaptive gate policy

9.1 The strict-vs-soft decision

Gate strictness must scale with the cost of an undetected error.

Error type	Cost	Gate
Wrong baseline value written to disk	High — calibration breaks	Strict
Wrong source URL in <code>sources_registry.yaml</code>	High — provenance audit fails	Strict
Off-tone narration in an insight	Low — user reads, has agency	Soft
Suboptimal scenario perturbation manifest	Medium — exploratory failure acceptable	Soft (with N=2 retry)
Wrong interview question difficulty	Medium — user can request a different one	None (calibration-locked)

9.2 Strict gate composition

Strict gates use the full sanitize/synthesize stack:

1. Schema sanitizer (deterministic, ~\$0.0001).
2. Provenance sanitizer (deterministic, ~\$0.0001).
3. Source-tier sanitizer (deterministic; checks `sources_registry.yaml` trust tier).
4. Bound sanitizer (deterministic; checks value is within plausible range from baselines).
5. Synthesizer (Opus 4.7 with `thinking_budget=16000`, ~\$0.10).
6. PreToolUse hook blocks if confidence < 0.85 OR severity ≥ high.
7. `ctx.elicit()` for human approval on borderline cases.

Cost per write: ~\$0.10 + sanitizer overhead. Latency: ~5-10s.

This is appropriate for high-stakes writes (calibrated baselines). It is over-engineering for narration.

9.3 Soft gate composition

Soft gates use a lighter stack:

1. PostToolUse hook annotates output with `{disputed, confidence, disputed_by}`.
2. No PreToolUse block (soft mode allows the write).
3. Optional: LLM-as-judge for ambiguous content, but not gating.

Cost per write: ~\$0.001 (annotation only). Latency: <1s.

The user reads the output. If it's wrong, they push back. The cost of an off-tone insight is low; the cost of always-strict is high (latency, \$\$, false positives).

9.4 Multi-stage gates

Some flows benefit from chained gates. Example: scenario synthesis (M15).

1. **Soft gate stage 1:** Compiler outputs natural-language manifest. Evaluator scores against rubric. If score ≥ 0.7 , proceed. If score < 0.7 , retry up to $N=2$ times.
2. **Strict gate stage 2:** Once a manifest is produced, validate it parses, validate target paths exist in the engine's perturbation handler, validate values are within bounds.

Stage 1 is exploratory (soft). Stage 2 is structural (strict). Composing them lets agents iterate creatively while preventing structurally-invalid outputs from reaching the engine.

10. Concrete code patterns

10.1 FastMCP server with mount() composition

```
# infra/server.py
import os
from fastmcp import FastMCP
from fastmcp.transports import StreamableHTTPSessionManager
from infra.auth import oauth_provider
from infra.eventstore import RedisEventStore

# Sub-servers (each is its own FastMCP() instance in its own
module):
    from baselines.server import baselines_server    #
    FastMCP("baselines")
    from curation.server import curation_server      #
    FastMCP("curation")
    from engine.server import engine_server          #
    FastMCP("engine_compiler")
    from insights.server import insights_server      #
    FastMCP("insights")

front_door = FastMCP("Reels Hybrid", auth=oauth_provider)

# Mount under layer-numbered prefixes
front_door.mount("l4", baselines_server)
front_door.mount("l5", curation_server)
front_door.mount("l6", engine_server)
front_door.mount("l8", insights_server)

# Stateful session manager with Redis EventStore
session_manager = StreamableHTTPSessionManager(
    stateless=False,
    event_store=RedisEventStore(os.environ["REDIS_URL"]),
)

if __name__ == "__main__":
    front_door.run(
        transport="streamable-http",
        session_manager=session_manager,
        host="0.0.0.0",
        port=int(os.environ.get("PORT", 8000)),
    )
```

10.2 Sanitize tool (deterministic)

```
# baselines/tools/sanitize_schema.py
from fastmcp import Context
from baselines.schemas import ExtractedMetric, SanitizerFlag
import jsonschema

@baselines_server.tool(annotations={"readOnlyHint": True})
async def sanitize_schema(extracted: ExtractedMetric, ctx: Context)
-> list[SanitizerFlag]:
    """Validate ExtractedMetric against the canonical schema.
    Deterministic; no LLM."""
    flags = []

    # Schema validation
    try:
        jsonschema.validate(extracted.dict(), CANONICAL_SCHEMA)
    except jsonschema.ValidationError as e:
        flags.append(SanitizerFlag(
            kind="schema",
            severity="high",
            evidence=[str(e)],
            span=None,
            suggested_fix="Conform to canonical schema",
        ))

    # Bound check
    if extracted.unit == "pct" and not (0 <= extracted.value <=
100):
        flags.append(SanitizerFlag(
            kind="bound",
            severity="high",
            evidence=[f"value={extracted.value} outside [0, 100] for
pct"],
            span=None,
            suggested_fix=None,
        ))

    # Provenance check
    if not extracted.source.url and not extracted.source.doc_title:
        flags.append(SanitizerFlag(
            kind="provenance",
            severity="critical",
            evidence=["No URL or doc_title in source"],
            span=None,
            suggested_fix="Add source attribution",
        ))

    return flags
```

10.3 Synthesizer pattern (sole writer)

```
# baselines/tools/synthesize_baseline.py
import aiofiles
import yaml
from fastmcp import Context
from baselines.schemas import ExtractedMetric, SanitizerFlag,
SynthesizerDecision
from calibration.lock import verify_lock_against_current_state

@baselines_server.tool()
async def synthesize_baseline(
    extracted: ExtractedMetric,
    sanitizer_flags: list[SanitizerFlag],
    target_file: str,
```

```

        target_path: str,
        ctx: Context,
    ) -> SynthesizerDecision:
        """Sole writer to baselines/data/. Calls Opus 4.7 judge, writes
on accept."""

        # Pre-write: verify calibration lock still valid
        valid, drifted = verify_lock_against_current_state()
        if not valid:
            return SynthesizerDecision(
                decision="reject",
                confidence=0.0,
                rationale=f"Calibration drift in {drifted}",
                accepted_fixes=[],
                unresolved_disputes=["calibration_drift"],
            )

        # Calibration-impact pre-check
        if would_change_locked_hash(target_file, target_path,
extracted.value):
            approve = await ctx.elicit(
                f"Change to {target_file}::{target_path} would
invalidate calibration. "
                f"Approve and re-run calibration after?"
            )
            if not approve:
                return SynthesizerDecision(
                    decision="reject",
                    confidence=0.0,
                    rationale="User declined calibration-invalidating
change",

                    accepted_fixes=[],
                    unresolved_disputes=[],
                )

        # Call Opus 4.7 judge
        judge_response = await ctx.sample(
            model="claude-opus-4-7",
            messages=[
                {"role": "user", "content":
build_judge_prompt(extracted, sanitizer_flags)}
            ],
            thinking={"type": "enabled", "budget_tokens": 16000},
        )
        decision = parse_judge_response(judge_response)

        # Persist decision (always, regardless of accept/reject)
        await persist_synthesis(decision, extracted, sanitizer_flags)

        # Write only on accept with confidence ≥ floor
        if decision.decision == "accept" and decision.confidence >=
0.85:
            await write_yaml_atomic(target_file, target_path,
extracted.value, extracted.source)

        return decision

    async def write_yaml_atomic(file_path: str, target_path: str, value,
source):
        """Atomic yaml write. Synthesizer is the sole caller of this
function."""
        async with aiofiles.open(file_path, "r") as f:
            content = yaml.safe_load(await f.read())
            set_path(content, target_path, {"value": value, "source":
source.url, "provenance": "earnings"})
        async with aiofiles.open(file_path + ".tmp", "w") as f:

```

```

        await f.write(yaml.dump(content))
    os.rename(file_path + ".tmp", file_path)

```

10.4 PreToolUse hook enforcing layer policy

```

# orchestrator/hooks.py
def make_layer_isolation_hook():
    """Block agentic tool calls that try to write to deterministic-
    layer state."""
    DETERMINISTIC_PATHS = ("params/", "algorithms/", "engine/",
    "metrics/", "calibration/")

    async def hook(tool_name, tool_input, context):
        # Allow read-only tools
        if context.tool_annotations.get("readOnlyHint"):
            return None

        # For non-synthesizer tools, deny writes to deterministic
        layers
        target = tool_input.get("target_file") or
        tool_input.get("path") or ""
        if not tool_name.endswith("synthesize"):
            for path_prefix in DETERMINISTIC_PATHS:
                if target.startswith(path_prefix):
                    return {
                        "hookSpecificOutput": {
                            "hookEventName": "PreToolUse",
                            "permissionDecision": "deny",
                            "permissionDecisionReason": (
                                f"Tool {tool_name} attempted write
                                to deterministic layer "
                                f"{path_prefix}. Only synthesizer
                                tools may write to canonical state."
                            ),
                        },
                    }
            }
        return None

    return hook

```

10.5 Calibration verify wrapper

```

# calibration/lock.py
import functools
from calibration.lock import verify_lock_against_current_state

def requires_calibration_unlocked(f):
    """Decorator: aborts agentic operation if calibration lock is
    invalid."""
    @functools.wraps(f)
    async def wrapper(*args, **kwargs):
        valid, drifted = verify_lock_against_current_state()
        if not valid:
            raise CalibrationLockError(
                f"Calibration drift detected in {drifted}. "
                f"Re-run python -m calibration.runner and re-lock
                before proceeding."
            )
        return await f(*args, **kwargs)
    return wrapper

# Usage on every agentic synthesizer:
@requires_calibration_unlocked
async def synthesize_baseline(...):
    ...

```

11. What's likely to change

This document is a snapshot. The MCP spec, FastMCP, and the Claude Agent SDK are evolving. Here's what to monitor:

MCP spec updates. The spec is at v0.x as of April 2026. Watch for v1.0 stabilization. Likely changes: tighter sampling protocol semantics, formal capability negotiation, possibly built-in resumability.

FastMCP API surface. `mount()` is stable; Client chaining patterns may get explicit "do not use for sampling" warnings in future versions. The `StreamableHTTPSessionManager` API may consolidate.

Claude Agent SDK. Hooks (`PreToolUse`, `PostToolUse`, `UserPromptSubmit`) are a recent addition (2025) and the API may add new hook points or change parameter shapes. The deliberation contract pattern in this document is built around current hook semantics.

Prompt cache TTL. Current default is 5 minutes; explicit `"ttl": "1h"` is the workaround. If Anthropic raises the default or makes it user-configurable, the explicit setting becomes redundant.

Citations + Structured Outputs. The 400 error when combining these is a current limitation. Future API versions may allow combination; until then, the two-call pattern is necessary.

Replit deployment surface. Reserved VM vs Autoscale economics may shift. The architectural recommendations here assume Reserved VM is appropriate for sampling-heavy workloads; if Autoscale gains sampling-friendly cold-start (sub-second), that recommendation may relax.

Multi-agent patterns. Anthropic's multi-agent research (2024–2025) suggested patterns for orchestrator-with-workers; this document recommends caution about premature multi-agent for tightly-coupled tasks. As patterns mature, the guidance may broaden.

The core architectural decisions — single front-door via `mount()`, `sanitize/synthesize`, agents only on Layers 4/5/6/8, calibration-as-hard-gate — are unlikely to change. They reflect the structure of the MCP protocol and the cost-of-error analysis, not specific API surfaces.

Appendix: AGENTIC_ARCHITECTURE_INDEX.md skeleton

When Claude Code reads this PDF during the M12 prelude, it generates `docs/AGENTIC_ARCHITECTURE_INDEX.md` as a structural summary. The index has these sections (Claude Code populates each from this PDF's content):

1. **Agentic vs deterministic layer map.** Cite §3 of this PDF. Per-layer agentic mode (strict/soft/none).
2. **Sanitize/synthesize gate policy.** Cite §2 and §9. Strict for L4/L5; soft for L6 scenario, L8 insights.
3. **FastMCP composition rule.** Cite §1. `mount()` vs chained Client; why chained breaks sampling.
4. **Replit deployment constraints.** Cite §5. Stateful HTTP, Redis EventStore, Postgres-not-SQLite, fork subagent flag, prompt cache TTL.
5. **Anti-pattern catalog.** Cite §7. The 11 do-nots with the file/layer where each surfaces.
6. **Sampling channel preservation.** Cite §8. What breaks, what

preserves.

The index is one-page, link-heavy, fast-lookup. The PDF is the authoritative explanation.